

Отчёта по лабораторной работе 4

Имитационное моделирование

Нджову Нелиа

Содержание

Список иллюстраций

Список таблиц

1 Цель работы

Цель этой лабораторной работы использовать агентную модель для моделирования SIR модель.

2 Задание

1. Базовый уровень

— Запустите модель с параметрами по умолчанию. — Постройте график динамики численности S , I , R . — Определите базовое репродуктивное число R_0 по формуле $R_0 = \frac{\beta}{\gamma}$, где $\gamma = \frac{1}{infection_period}$. — Сравните с наблюдаемой динамикой.

2. Исследование порога

— Найдите минимальное значение β , при котором возникает эпидемия (пик $I > 5\%$ популяции). — Сравните с теоретическим порогом $R_0 = 1$.

3. Эффект гетерогенности

— Задайте разные значения β для разных городов. — Как это влияет на общую динамику? — Постройте графики для каждого города отдельно.

4. Миграция

— Исследуйте влияние интенсивности миграции на скорость распространения инфекции из одного города в другие. — При какой интенсивности время до пика минимально?

5. Карантинные меры — Модифицируйте модель, добавив возможность закрытия города (обнуление миграции из него) при превышении порога заболеваемости. — Оцените эффективность такой меры.

6. Оптимизация — Используя оптимизацию, найдите параметры, которые минимизируют общее число умерших при сохранении пика заболеваемости ниже 30%.

3 Теоретическое введение

Создадим агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-Infectious-Recovered). Модель будет реализована с использованием пакета Agents.jl. В отличие от классической модели на дифференциальных уравнениях, агентный подход позволит учесть индивидуальные характеристики, пространственную структуру и стохастичность процессов.

3.1 Классическая модель SIR

Модель SIR, предложенная Кермаком и Маккендриком в 1927 году, описывает динамику эпидемии в популяции, разделённой на три группы:

— S (Susceptible) — восприимчивые к заболеванию индивиды; — I (Infectious) — инфицированные, способные заражать восприимчивых; — R (Recovered) — выздоровевшие (или умершие), получившие иммунитет и более не участвующие в распространении.

Классическая модель описывается системой обыкновенных дифференциальных уравнений:

$$\frac{ds}{dt} = \frac{\beta * S * I}{N}, \frac{dI}{dt} = \frac{\beta * S * I}{N} - \gamma * I, \frac{dR}{dt} = \gamma * I$$

где β — коэффициент передачи инфекции, γ — скорость выздоровления.

Ограничения классического подхода: Однородность популяции, Отсут-

ствие пространственной структуры, Детерминированность и Непрерывность

Преимущества агентного подхода: Агентное моделирование позволяет преодолеть эти ограничения:

- Каждый индивид моделируется отдельно с уникальными характеристиками.
- Взаимодействия происходят локально в пространстве или социальной сети.
- Процессы носят стохастический характер.
- Можно учитывать гетерогенность контактов, мобильность, меры контроля.

3.2 Агенты

Каждый агент — человек, обладающий следующими свойствами:

- `status::Symbol` — состояние (:S, :I, :R);
- `days_infected::Int` — количество дней с момента заражения (для инфицированных).

3.3 Пространство состояний

Агенты размещаются на графе, где узлы представляют локации (города, районы), а рёбра — возможные перемещения между ними. Такой подход позволяет моделировать метапопуляционную динамику и миграцию.

3.4 Параметры модели

- `Ns` — вектор численности населения по локациям;
- β_{und} — вероятность передачи для невыявленных случаев;
- β_{det} — вероятность передачи для выявленных случаев;
- `infection_period` — длительность заболевания (дней);
- `detection_time` — время до выявления заболевания;
- `death_rate` — вероятность летального исхода;
- `reinfection_probability` — вероятность повторного

заражения; — `migration_rates` — матрица вероятностей миграции между локациями.

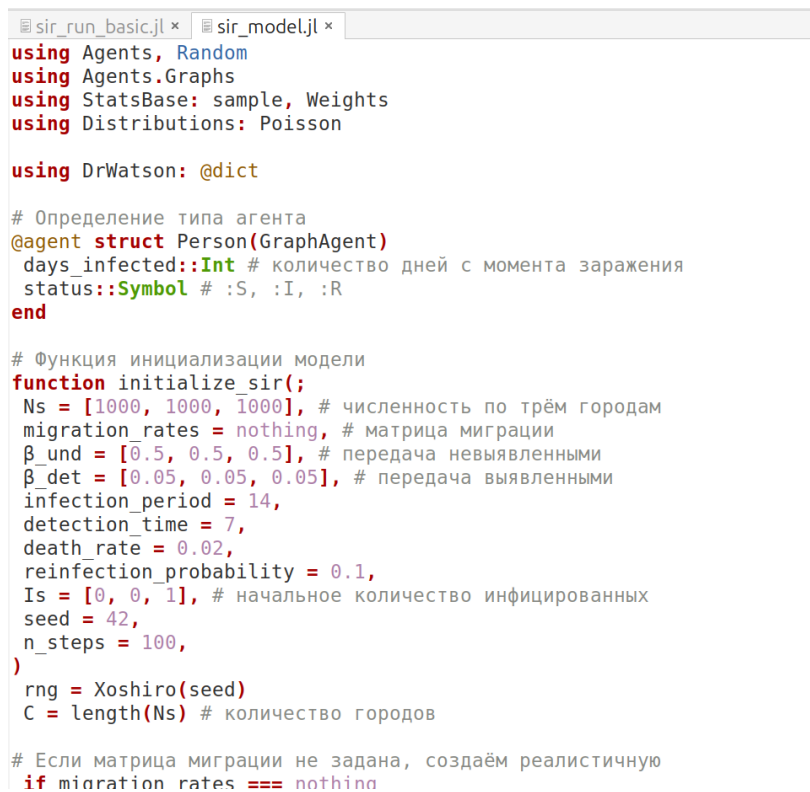
3.5 Динамика

На каждом шаге (соответствует одному дню) для каждого агента выполняются следующие процессы:

- Миграция — агент может переместиться в другую локацию с вероятностью, заданной матрицей `migration_rates`.
- Передача инфекции — если агент инфицирован, он может заразить восприимчивых агентов в той же локации. Количество заражённых за день определяется пуассоновским процессом с интенсивностью, зависящей от статуса выявления (β_{und} или β_{det}).
- Обновление счётчика — для инфицированных увеличивается `days_infected`.
- Выздоровление или смерть — если `days_infected` достиг `infection_period`, агент либо выздоравливает (переходит в R), либо умирает (удаляется из модели) с вероятностью `death_rate`. Выздоровевшие могут быть повторно инфицированы с вероятностью `reinfection_probability`.

4 Выполнение лабораторной работы

Я создала файл `sir_model.jl` и копировала в него скрипт из лабораторного руководства. скрипт определяет основную модель SIR (агенты, распространение болезней, миграция, выздоровление и смерть), которую все сценарии эксперимента используют в качестве механизма моделирования. Я запускаю его, и все работает идеально(рис.1).



```
sir_run_basic.jl × sir_model.jl ×
using Agents, Random
using Agents.Graphs
using StatsBase: sample, Weights
using Distributions: Poisson

using DrWatson: @dict

# Определение типа агента
@agent struct Person(GraphAgent)
    days_infected::Int # количество дней с момента заражения
    status::Symbol # :S, :I, :R
end

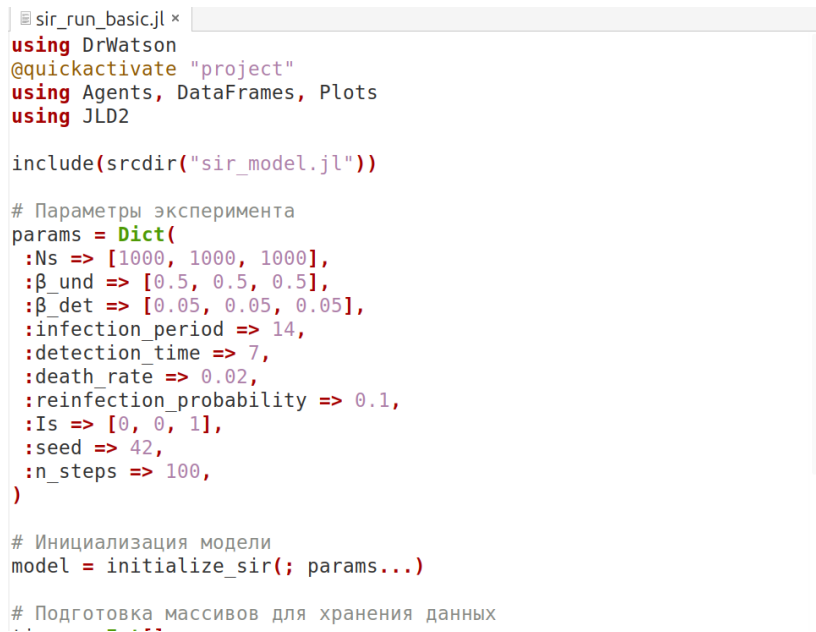
# Функция инициализации модели
function initialize_sir(;
    Ns = [1000, 1000, 1000], # численность по трём городам
    migration_rates = nothing, # матрица миграции
    β_und = [0.5, 0.5, 0.5], # передача не выявленными
    β_det = [0.05, 0.05, 0.05], # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1], # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
)
    rng = Xoshiro(seed)
    C = length(Ns) # количество городов

    # Если матрица миграции не задана, создаём реалистичную
    if migration_rates === nothing
```

Рисунок 4.1: Основной модель SIR

4.1 1. Базовый уровень

Я создала файл `sir_run_basic.jl` и копирую в него скрипт из лабораторного руководства. Скрипт запускает один эксперимент с фиксированными параметрами (по умолчанию) и сохраняет динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса.(рис.2)

A screenshot of a code editor window titled 'sir_run_basic.jl'. The code is written in Julia and defines parameters for a simulation. It includes comments in Russian. The code is as follows:

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2

include(scrdir("sir_model.jl"))

# Параметры эксперимента
params = Dict{
  :Ns => [1000, 1000, 1000],
  :β_und => [0.5, 0.5, 0.5],
  :β_det => [0.05, 0.05, 0.05],
  :infection_period => 14,
  :detection_time => 7,
  :death_rate => 0.02,
  :reinfection_probability => 0.1,
  :Is => [0, 0, 1],
  :seed => 42,
  :n_steps => 100,
}

# Инициализация модели
model = initialize_sir(; params...)

# Подготовка массивов для хранения данных
```

Рисунок 4.2: Базовый эксперимент

Я преобразую код в литературный стиль(рис.3)

```

# # Агентная модель SIR: Базовый эксперимент
#
# Надо Запустить базовый эксперимент с моделью SIR, проанализировать
# динамику эпидемии и сохранить результаты.
#
# Инициализируем проект DrWatson и подключаем необходимые пакеты.

using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2

# Подключаем модуль с определением модели SIR
include(scrdir("sir_model.jl"))

# ## Параметры эксперимента
#
# Определяем параметры модели:
# - **Ns**: численность населения в трёх городах
# - **β_und**: вероятность заражения невыявленными больными
# - **β_det**: вероятность заражения выявленными больными
# - **infection_period**: длительность заболевания
# - **detection_time**: время до выявления
# - **death_rate**: вероятность летального исхода
# - **reinfection_probability**: вероятность повторного заражения
# - **Is**: начальное количество инфицированных
# - **seed**: зерно генератора случайных чисел
# - **n_steps**: количество дней симуляции

params = Dict{
    :Ns => [1000, 1000, 1000],
    :β_und => [0.5, 0.5, 0.5],

```

Рисунок 4.3: Литературный стиль

Я запустила его, и все заработало, я получила график ниже. На графике мы можем видеть, как работает модель sir: сначала инфицированных людей нет, но с течением времени заболевает все больше и больше людей, пока все они не будут инфицированы, некоторые не вылекутся, а другие не умрут(рис.4)



Рисунок 4.4: График модель SIR

После этого я создала текст на основе литературного кода: чистого кода, записной книжки jupyter и документации в формате Quarto. Я запускала все ячейки в файле jupyter notebook, и все работает хорошо(рис.5)

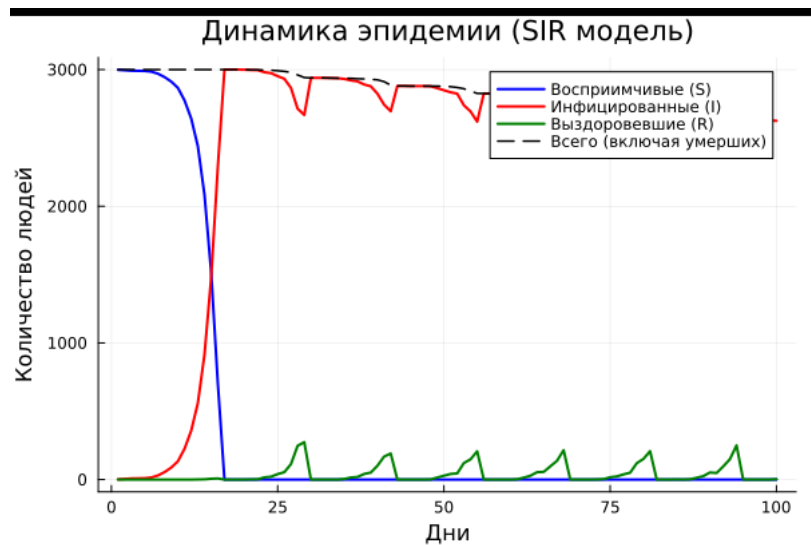


Рисунок 4.5: создание файлов

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/sir_run_basic.jl.qmd`

5 Агентная модель SIR: Базовый эксперимент

Надо Запустить базовый эксперимент с моделью SIR, проанализировать динамику эпидемии и сохранить результаты.

Инициализируем проект DrWatson и подключаем необходимые пакеты.

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using Plots
gr(fmt=:png)
using JLD2
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Подключаем модуль с определением модели SIR

```
include(sourcedir("sir_model.jl"))
```

```
total_count (generic function with 1 method)
```

5.1 Параметры эксперимента

Определяем параметры модели: - **Ns**: численность населения в трёх городах - **β_{und}** : вероятность заражения невыявленными больными - **β_{det}** : вероятность заражения выявленными больными - **infection_period**: длительность заболевания - **detection_time**: время до выявления - **death_rate**: вероятность летального исхода - **reinfection_probability**: вероятность повторного заражения - **Is**: начальное количество инфицированных - **seed**: зерно генератора случайных чисел - **n_steps**: количество дней симуляции

```
params = Dict(
  :Ns => [1000, 1000, 1000],
  : $\beta_{und}$  => [0.5, 0.5, 0.5],
  : $\beta_{det}$  => [0.05, 0.05, 0.05],
  :infection_period => 14,
  :detection_time => 7,
  :death_rate => 0.02,
  :reinfection_probability => 0.1,
  :Is => [0, 0, 1],
  :seed => 42,
  :n_steps => 100,
)
```

Dict{Symbol, Any} with 10 entries:

:Is	=> [0, 0, 1]
:seed	=> 42
:infection_period	=> 14
:n_steps	=> 100
: β_{und}	=> [0.5, 0.5, 0.5]
:Ns	=> [1000, 1000, 1000]


```

:detection_time      => 7
:reinfection_probability => 0.1
:[]_det              => [0.05, 0.05, 0.05]
:death_rate          => 0.02

```

Создаём модель с заданными параметрами.

```
model = initialize_sir(; params...)
```

StandardABM with 3000 agents of type Person

agents container: Dict

space: GraphSpace with 3 positions and 3 edges

scheduler: fastest

properties: C, infection_period, []_und, Ns, migration_rates, detection_time, reinfection

Создаём массивы для хранения динамики популяций на каждом шаге.

```

times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

```

```
Int64[]
```

Выполняем пошаговую симуляцию на n_steps дней. На каждом шаге собираем статистику о состоянии популяции.

```

println("Запуск симуляции на $(params[:n_steps]) дней...")

for step = 1:params[:n_steps]

```

```

Agents.step!(model, 1) # Выполняем один шаг моделирования

push!(times, step)
push!(S_vals, susceptible_count(model))
push!(I_vals, infected_count(model))
push!(R_vals, recovered_count(model))
push!(total_vals, total_count(model))

if step % 10 == 0 # Выводим прогресс каждые 10 шагов
    println("  День $step: I = $(I_vals[end]), R = $(R_vals[end])")
end
end

println("Симуляция завершена.")

```

Запуск симуляции на 100 дней...

```

День 10: I = 134, R = 0
День 20: I = 2999, R = 0
День 30: I = 2939, R = 0
День 40: I = 2824, R = 101
День 50: I = 2849, R = 28
День 60: I = 2822, R = 0
День 70: I = 2763, R = 0
День 80: I = 2566, R = 151
День 90: I = 2636, R = 52
День 100: I = 2625, R = 3

```

Симуляция завершена.

Преобразуем собранные данные в удобный табличный формат.

```
agent_df = DataFrame(  
    time = times,  
    susceptible = S_vals,  
    infected = I_vals,  
    recovered = R_vals  
)  
  
model_df = DataFrame(  
    time = times,  
    total = total_vals  
)
```

	time	total
	Int64	Int64
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000
10	10	3000
11	11	3000
12	12	3000
13	13	3000
14	14	3000
15	15	3000
16	16	3000
17	17	3000
18	18	3000
19	19	3000
20	20	2999
21	21	2998
22	22	2998
23	23	2997
24	24	2995
25	25	2992
26	26	2987
27	27	2979
28	28	2961
29	29	2942
30	30	2939
...	...	...

Выводим итоговые показатели

```
println("Общая численность населения: ${total_vals[end]}")
println("Переболело: ${R_vals[end]} человек (${round(R_vals[end]/1000*100, digits=1)})")
println("Умерло: ${total_vals[1] - total_vals[end]} человек")
println("Максимальное число инфицированных: ${maximum(I_vals)}")
```

Общая численность населения: 2628

Переболело: 3 человек (0.3%)

Умерло: 372 человек

Максимальное число инфицированных: 3000

5.2 Визуализация результатов

Строим график динамики эпидемии: - Синяя линия: восприимчивые (S) -
Красная линия: инфицированные (I) - Зелёная линия: выздоровевшие (R) -
Пунктирная линия: общая численность (с учётом умерших)

```
plot(
    agent_df.time,
    agent_df.susceptible,
    label = "Восприимчивые (S)",
    xlabel = "Дни",
    ylabel = "Количество людей",
    title = "Динамика эпидемии (SIR модель)",
    linewidth = 2,
    color = :blue,
)
plot!(agent_df.time, agent_df.infected, label = "Инфицированные (I)", linewidth = 2,
```